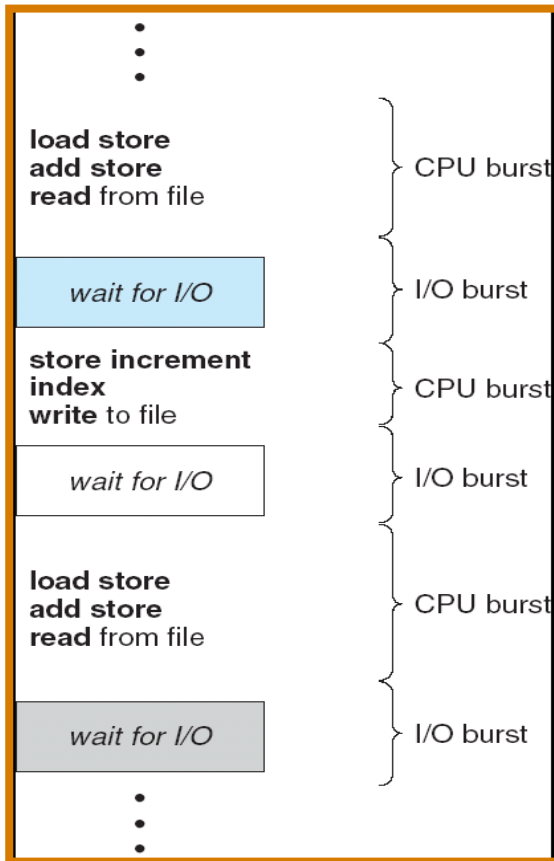


CPU Scheduling

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms

Basic Concepts



- CPU-I/O Burst Cycle
- Process execution consists of a cycle of CPU execution and I/O wait.
- Processes alternate between these two states.
- Process execution begins with CPU burst, followed by an I/O burst and so on.
- Last CPU burst will end with a system request to terminate execution.

CPU Scheduler

The **CPU scheduler** (short term scheduler) selects from among the processes in ready queue, and allocates a CPU core to one of them

- Queue may be ordered in various ways

CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state
2. Switches from running to ready state Process chalte chalte beech m nikal sakta h
3. Switches from waiting to ready I/O karne k baad wapis ayega toh fir scheduling hoga
4. Terminates

For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution. (non-Preemptive)

For situations 2 and 3, however, there is a choice (Preemptive).

Preemptive and Nonpreemptive Scheduling

? Non-preemptive Scheduling

? Once CPU has been allocated to a process, the process keeps the CPU until

? Process exits OR

? Process switches to waiting state

? Preemptive Scheduling

? Process can be interrupted and must release the CPU.

? Need to coordinate access to shared data

Dispatcher

Dispatcher module gives control of the CPU to the process selected by the short-term scheduler. This involves:

- switching context
- switching to user mode
- jumping to the proper location in the user program to restart that program

Dispatch Latency:

- Time it takes for the dispatcher to stop one process and start another running.
- Dispatcher must be fast.

Scheduling Criteria

CPU Utilization

- ❑ Keep the CPU and other resources as busy as possible. In real system it should range from 40 percent for lightly used systems to 90 percent for heavily used systems

Throughput

- ❑ The number of processes that complete their execution per time unit.

Turnaround time

- ❑ The interval from the time of submission of a process to the time of completion.
- ❑ Its the sum of periods spent waiting to get memory, in ready queue, execution time, and doing I/O.

Scheduling Criteria...

Waiting time

- ❑ amount of time a process has been waiting in the ready queue.
- ❑ Sum of the periods spent waiting in the ready queue.

Response Time (in a time-sharing environment)

- ❑ amount of time it takes from when a request was submitted until the first response is produced, NOT output.

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Scheduling Algorithms

- First-Come First-Serve (FCFS) (Non-preemptive)
- Shortest Job First (SJF)
 - Non-preemptive
 - Pre-emptive
- Priority
 - Non-preemptive
 - Pre-emptive
- Round-Robin(Pre-emptive)
- Multilevel Queue
- Multilevel Feedback Queue

First Come First Serve Scheduling

- Policy: Process that requests the CPU FIRST is allocated the CPU FIRST.
 - FCFS is a non-preemptive algorithm.
- Implementation - using FIFO queues
 - Incoming process is added to the tail of the queue.
 - Process selected for execution is taken from head of queue.
- Performance metric - Average waiting time in queue.
- Example
- Convoy Effect

Suppose the arrival order for the processes is

P1, P2, P3

Waiting time P1 = 0; P2 = 24; P3 = 27;

Average waiting time

$$(0+24+27)/3 = 17$$

Average completion time $(24+27+30)/3 = 27$

Process	Burst Time
P1	24
P2	3
P3	3

Gantt Chart for Schedule



FCFS Scheduling ...

Suppose that the processes arrive in the order:

P_2, P_3, P_1

The Gantt chart for the schedule is:



Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$

Average waiting time: $(6 + 0 + 3)/3 = 3$

Much better than previous case 

Convoy effect - short process behind long process

- Consider one CPU-bound and many I/O-bound processes

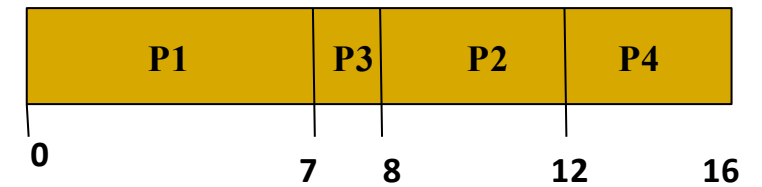
Shortest job First

- Associate with each process the length of its next CPU burst.
- Use these lengths to schedule the process with the shortest time.
- Two Schemes:
 - Scheme 1: Non-preemptive
 - Once CPU is given to the process it cannot be preempted until it completes its CPU burst.
 - Scheme 2: Preemptive
 - If a new CPU process arrives with CPU burst length less than remaining time of current executing process, preempt. Also called Shortest-Remaining-Time-First (SRTF).

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

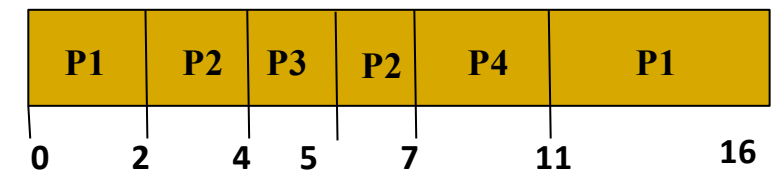
Non-Preemptive SJF Scheduling

Gantt Chart for Schedule



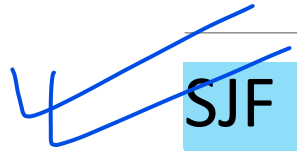
Preemptive SJF Scheduling

Gantt Chart for Schedule



Average waiting time =
 $(9+1+0+2)/4 = 3$

SJF...



SJF is optimal – gives minimum average waiting time for a given set of processes

Preemptive version called **shortest-remaining-time-first**

Disadvantages: Starvation for long jobs. Hard to predict future jobs.

How do we determine the length of the next CPU burst?

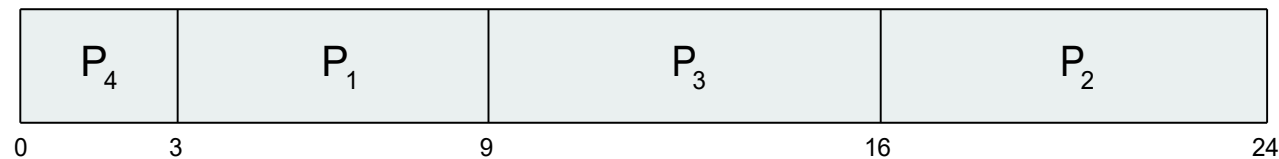
- Could ask the user
- Estimate

Example of SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

← Longest job ka starvation

SJF scheduling chart



$$\text{Average waiting time} = (3 + 16 + 9 + 0) / 4 = 7$$

Determining Length of Next CPU Burst

Can only estimate the length – should be similar to the previous one

- Then pick process with shortest predicted next CPU burst

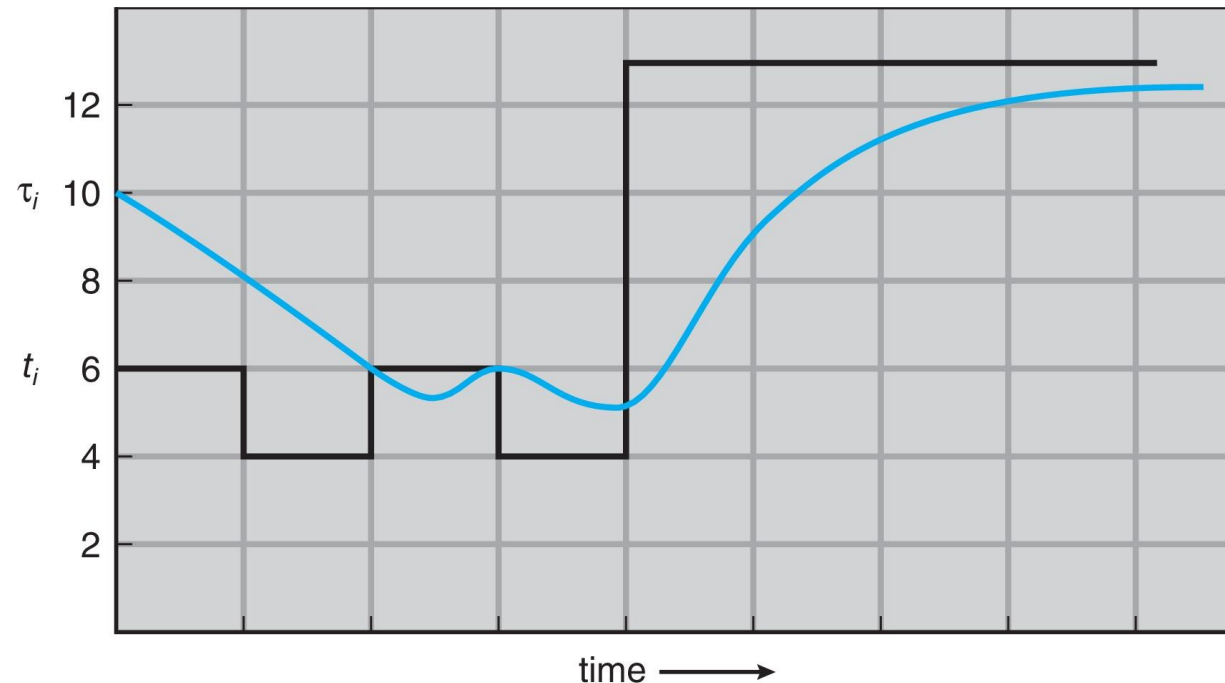
Can be done by using the length of previous CPU bursts, using exponential averaging

1. t_n = actual length of n^{th} CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. α , $0 \leq \alpha \leq 1$
4. Define:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n.$$

Commonly, α set to $\frac{1}{2}$

Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Examples of Exponential Averaging

$\alpha = 0$

- $\tau_{n+1} = \tau_n$
- Recent history does not count

$\alpha = 1$

- $\tau_{n+1} = \alpha t_n$
- Only the actual last CPU burst counts

If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

Since both α and $(1 - \alpha)$ are less than or equal to 1, each successor predecessor term has less weight than its predecessor

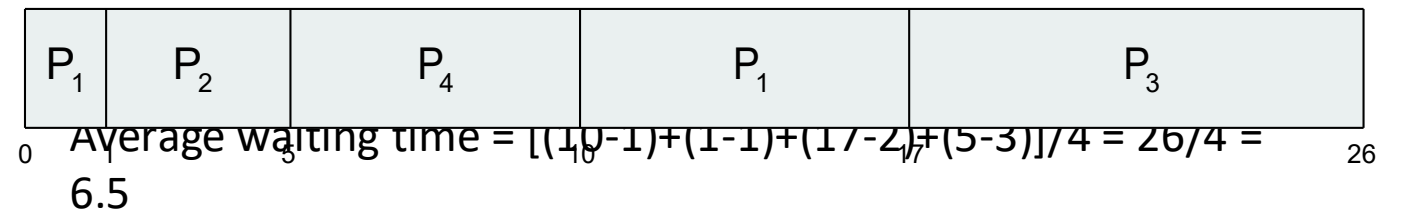
SRTF

Preemptive version of SJN

Whenever a new process arrives in the ready queue, the decision on which process to schedule next is redone using the SJN algorithm.

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Preemptive SJF Gantt Chart



Priority Scheduling

A priority value (integer) is associated with each process. Can be based on

- Cost to user
- Importance to user
- ✓ ◦ %CPU time used in last X hours.

CPU is allocated to process with the highest priority.

- Preemptive
- Nonpreemptive

? SJF is a priority scheme where the priority is the predicted next CPU burst time.

? Problem

? Starvation!! - Low priority processes may never execute.

? Solution

? Aging - as time progresses increase the priority of the process.

Aging is the solution to
starvation - Indefinite blocking

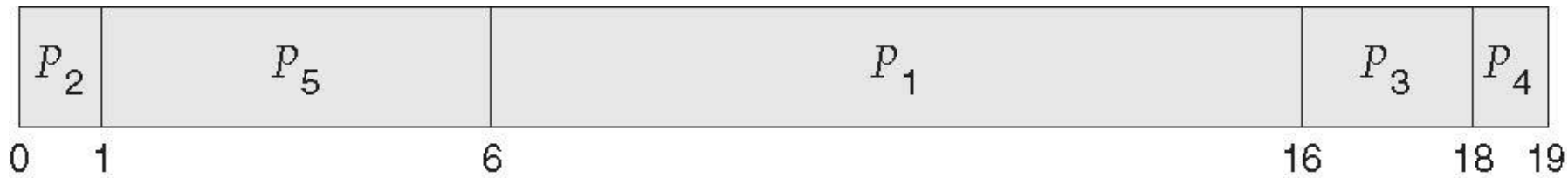


Example of Priority Scheduling

Process	Burst Time	Priority
---------	------------	----------

P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2 msec

Round Robin

Each process gets a small unit of CPU time

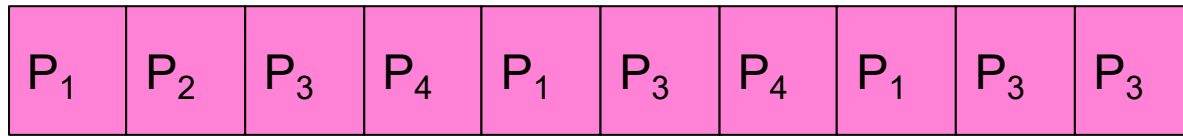
- ? Time quantum usually 10-100 milliseconds.
- ? After this time has elapsed, the process is preempted and added to the end of the ready queue.
- ? Timer interrupts every quantum to schedule next process

Performance

- q large $\Rightarrow$ FIFO (FCFS)
- q small $\Rightarrow$ RR

Round-Robin Pros and Cons:

- ? Better for short jobs.
- ? Context-switching time adds up for long jobs



Waiting time (time quantum 20)

$$P_1 = (68 - 20) + (112 - 88) = 72$$

$$P_2 = (20 - 0) = 20$$

$$P_3 = (28 - 0) + (88 - 48) + (125 - 108) = 85$$

$$P_4 = (48 - 0) + (108 - 68) = 88$$

Process	Burst Time
P_1	53
P_2	8
P_3	68
P_4	24

$$\text{Average waiting time} = (72 + 20 + 85 + 88) / 4 = 66\frac{1}{4}$$

$$\text{Average completion time} = (125 + 28 + 153 + 112) / 4 = 104\frac{1}{2}$$

Multilevel Queue

Ready queue is partitioned into separate queues, eg:

- **foreground** (interactive)
- **background** (batch)

Process permanently in a given queue

Each queue has its own scheduling algorithm:

- **foreground – RR**
- **background – FCFS**

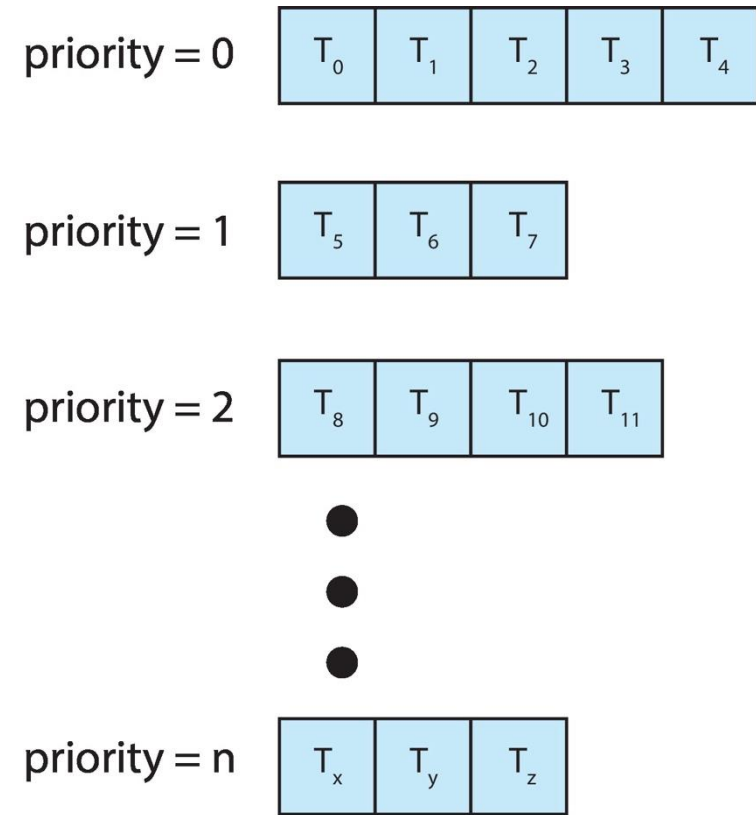
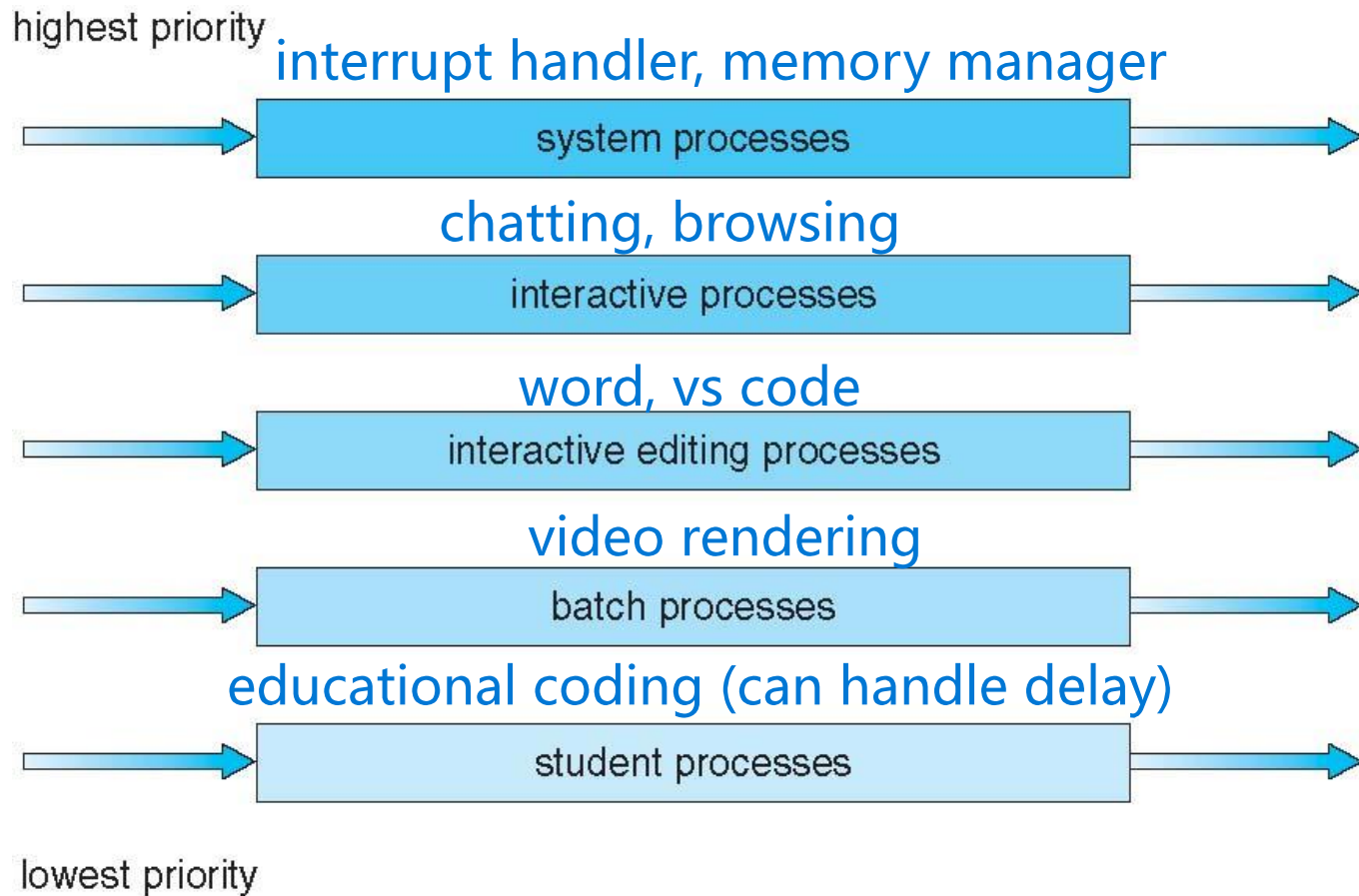
Scheduling must be done between the queues: **To avoid starvation (Indefinite blocking) AMONG QUEUES**

- **Fixed priority scheduling;** (i.e., **serve all from foreground then from background**). Possibility of starvation.
- **Time slice** – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., **80% to foreground in RR**
- **20% to background in FCFS**

Is Time Slicing the Solution to Starvation?
Yes — at inter-queue level

Because:
Each queue is guaranteed CPU time
No queue can completely block another queue
Hence, starvation between queues is avoided

Multilevel Queue Scheduling...



Multilevel Feedback Queue

A process can move between the various queues; aging can be implemented this way

Multilevel-feedback-queue scheduler defined by the following parameters:

- number of queues
- scheduling algorithms for each queue
- method used to determine when to upgrade a process
- method used to determine when to demote a process
- method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue...

A process can move between the various queues; aging can be implemented this way

Multilevel-feedback-queue scheduler defined by the following parameters:

- number of queues
- scheduling algorithms for each queue
- method used to determine when to upgrade a process
- method used to determine when to demote a process
- method used to determine which queue a process will enter when that process needs service

“Served FCFS” means processes are selected in order of arrival inside that queue.
The time limit (8 ms, 16 ms) controls how long they execute before possibly moving to a lower queue.

Example of Multilevel Feedback Queue

Three queues:

- Q_0 – RR with time quantum 8 milliseconds
- Q_1 – RR time quantum 16 milliseconds
- Q_2 – FCFS

Scheduling

- A new job enters queue Q_0 which is served FCFS
- When it gains CPU, job receives 8 milliseconds
- If it does not finish in 8 milliseconds, job is moved to queue Q_1
- At Q_1 job is again served FCFS and receives 16 additional milliseconds
- If it still does not complete, it is preempted and moved to queue Q_2

